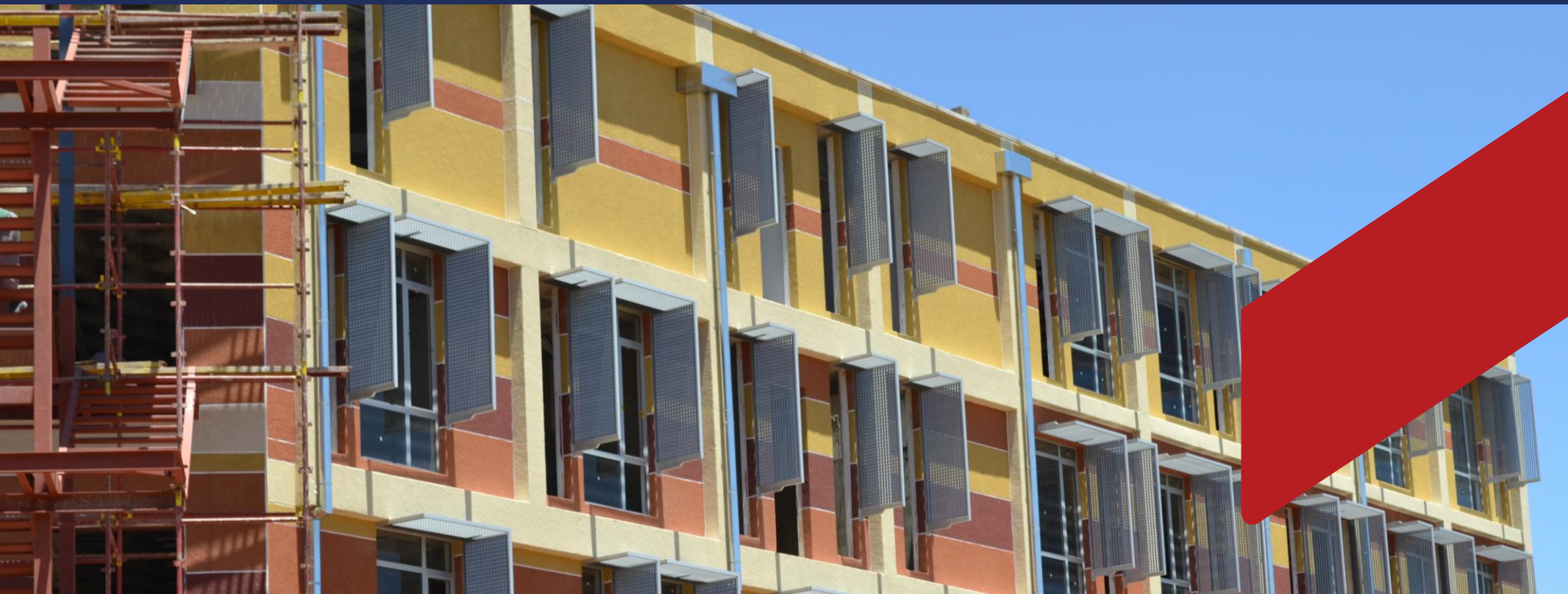
NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Lesson 3

Test System Architecture, Cases, and Coverage



NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Overview

System Testing (Test case Design, Building test cases, Test data mining, Test execution, Test Reporting, Defect Management, Test Coverage – Traceability matrix)

Automation Testing

Software quality concepts

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

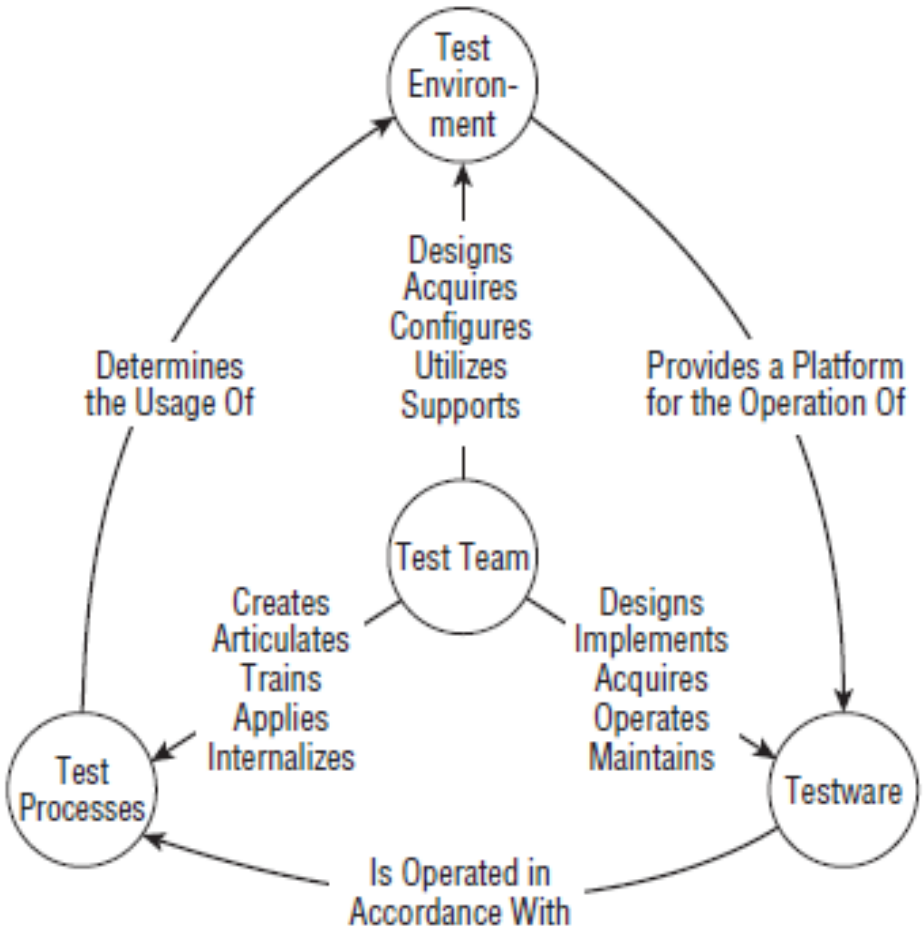
Test System Architecture and Engineering

- By test system, we mean the organizational capability for testing created by the testing processes, the testware, and the test environment.
- The testing processes include both written and unwritten procedures, checklists, and other agreements about the way the test team does its testing.
- The testware includes all those tools, documents, scripts, data, cases, tracking mechanisms, and so forth that the test team uses to do its testing.
- The test environment includes the hardware, software, networking and other infrastructure, paper and other supplies, facilities, lab, and so forth that the test team procures, installs, and configures the system under test in order to test it.
- A competent test team with a good test system can consistently provide effective and efficient test services to a (supportive) project.
- A good test system helps the tester focus her testing efforts on the key quality risks, and find, reproduce, isolate, describe, and manage the most important bugs in the software or hardware being tested, as well as capturing and analyzing key metrics.

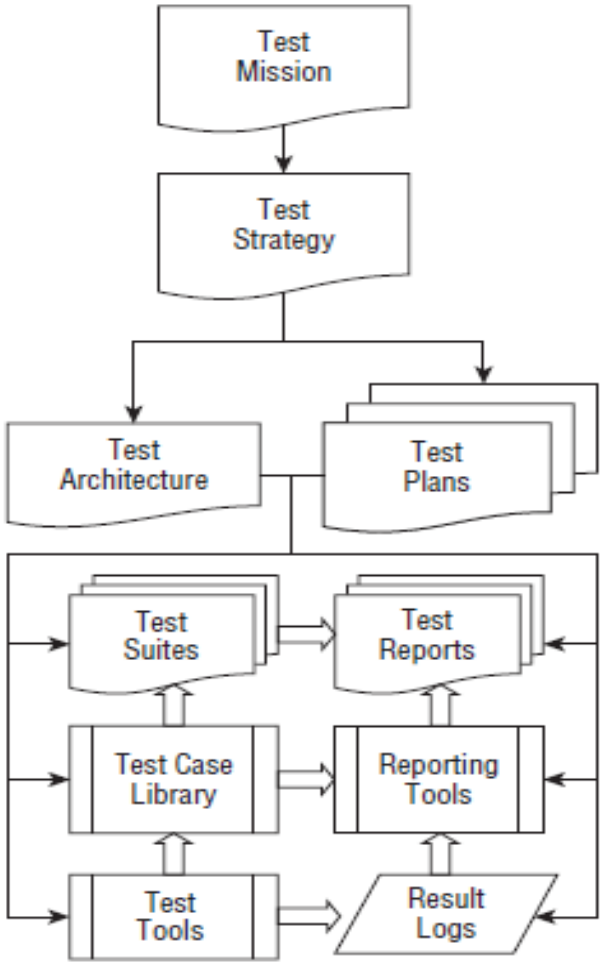
NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Composition of the Test System



The composition of the test system



A logical decomposition of typical testware components

Visual Representation of the Testware (*continues*)

- Testware often involves one or more test tools —for example, operating systems, scripting languages, GUI test automation systems, API test harnesses, oscilloscopes, or thermal chambers.
- Using these tools produces result logs, which are either created automatically from the tool or logged manually by the tester.
- Because the test team uses the tools to execute test cases, these tools support the test case library.
- The relationship between the two elements is many-to-many — that is, each tool can appear in multiple test cases, and any one test case can involve the use of multiple tools.
- Both the test case library and the result logs feed into reporting tools, with a greater or lesser degree of automation.
- Test engineers assemble test suites from the test case library; the relationship between cases and suites is also many-to-many.
- From the test suites and the reporting tools come test reports.
- Above these, the test architecture is a document that defines the design principles, the structure, and the tools that apply for the testware and test environment, as well as the interrelationships between the constituent pieces; it is project-independent, but it reflects the system under test.

Visual Representation of the Testware….

- The test plan harnesses the test system to the project.
- At the very top are the test mission and the test strategy.
- The test mission or test policy is a document that defines the goals of testing.

For example, do the goals of testing include finding defects? Building confidence? Reducing risk?

- In addition to these goals, the key process indicators or metrics are defined for determining whether the test team is meeting the goals.
- The test strategy document tells how — in a project-independent way — to go about fulfilling the goals.
- The test plan implements the test strategy in a way that fits the project and that achieves the mission as appropriate to the project.
- The test architecture implements the test strategy in a way that defines how the testware supports the guiding principles of testing in the test strategy.

The Action Components: Definitions

- The test case setup describes the steps needed to configure the test environment to run the test case.

Connecting a loopback device, making sure that a certain amount of disk space is available, and installing an application are examples of test case setup.

- The point of running a test case is to create a set of test conditions.
- The creation of these conditions allows the tester to assess the quality of the system in relation to particular risks to system quality and customer usage scenarios.
- Some of the conditions exist in parallel, and others exist in series.
- In the software world, running a test tool to consume all the buffers while submitting transactions is a specific condition that you might create as part of a performance test.
- In the hardware world, each cycle of the power switch is a test condition that might occur as part of a mechanical-life test.
- The test case teardown specifies the steps required to restore the test environment to a clean condition after execution of the test case (or test suite).
- Disconnecting a loopback device, deleting temporary files, and removing an application are examples of test case teardown.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

No Test System Is an Island: Testers and the Test System

- Considerations of consistency and simplicity bring us to the human side of the equation.
- No matter how automated your test system, it does not create, use, or maintain itself; people do.
- Testers must set up the test environment, start the tests, interpret the results, reproduce anomalies manually, isolate bugs by experimentation, and restore the test environment to a known state at the end of the testing.
- In addition, the testers will usually need to maintain the test system, especially the test cases and suites.
- This is especially true any time development adds new features to the product being tested, changes the definition of correct behavior for the product, or supports new configurations or environments.
- Maintenance is also necessary when you decide to enhance the test system to get better test coverage.
- Simplicity of the test system is also essential to usability.
- The best test systems are easy to learn and easy to use.
- When outside people join your team, take advantage of the opportunity to quiz them about how easy— or how difficult — it is to learn the test system.

Miscellaneous Best Practices and Principles for Quality Test Systems (continues)

- Often, test systems need documentation. This is especially true with automated test systems.
- The more complex the system is, the more documentation is needed.
- The document should cover three main topics.

First, you should tell the testers how to use the test system. Second, you should tell the testers how the test system works. Third, you should tell the testers why you built the test system the way you did.

- The test system should also document itself.
- Largely, this happens through internal consistency.
- Testers should find filenames, result logs, and the like easy to read, giving some clue about their purpose.
- Use intuitive and consistent variable names in programs and scripts, even if people won't read these files and logs during normal operation.
- When the test system doesn't work, a test engineer will have to figure out the problem, and obscure error messages such as SYS40: Wrong at 252.351.37.92 and files with names like TST001.TXT will make the job difficult.
- On the subject of files, try to ensure that your test system software doesn't become full of junk such as non-test-related documents, unidentifiable log files, output files associated with old bug isolation operations, and so forth.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Miscellaneous Best Practices and Principles for Quality Test Systems

- A well-designed test system promotes accountability.
- It should identify, usually through the logs and the result files, who ran specific tests.
- Using a generic tester login can cause frustration when you need to ask questions or investigate a problem.
- Testers should identify themselves when they log in so that the logs and the result files will bear date/time and owner information.
- Finally, if your test organization develops special tools, you'll want to avoid two common software development mistakes.
- First, don't forget the importance of configuration management and release engineering.
- You should have your testers check in, check out, and manage the software and documentation portions of the test system just like any other software.
- Second, remember to test the tests.
- A standard test plan and suite should exist for the test system.
- A tester should revise and execute the plan any time changes are made to the test system.
- You should employ quality assurance best practices such as reviews.

The Bricks and Mortar of the System: Test Cases

- In a sense, everything in the test system— the test processes, the test tools, the reporting formats, the test environment, and so forth— exists to support the execution of test cases.
- The test case —at least in my model of the test system— is where the test takes actions on the system under test, the tester supplies data to the system under test, and the system under test ends up in some state or states, with resulting outputs and behaviors that testers can compare to expected results.
- These three things — action, data, and expected results— are the heart of test execution, since they create the test conditions.

Creating Test Conditions

How exactly can testers use test cases to create test conditions?

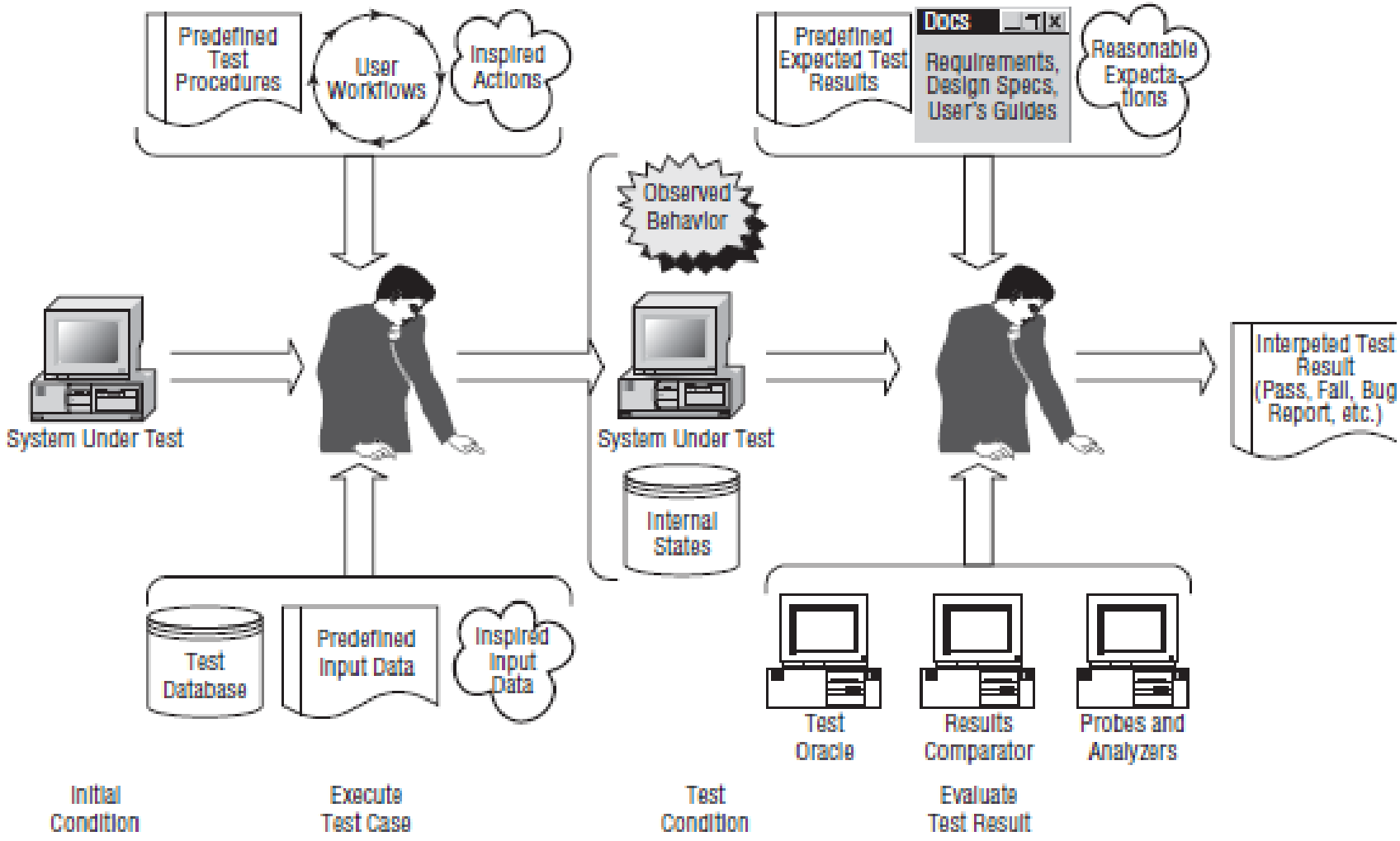
- At the highest level, we think of test cases as consisting of a sequence of actions, each action having potentially some associated test data and some associated expected result.
- Assume that we start at step one of a test case, the first action to take.
- The system is in whatever initial condition is reached from the test suite and test case setup steps.
- Following our test procedures, system documentation, or inspiration, we deliver some data to the system under test.
- Some of that data comes from persistent databases, some from various input devices, including from our creative fingers on keyboards, mice, and so forth.
- We have created the test condition.

The system under test responds. It processes this data, going through whatever computation steps occur. The system generates output and enters some subsequent state. The output, the subsequent state, and the way in which the system behaves during its response comprise the results. At this point, we evaluate the results, using automated tools, documented expected results, system documentation, and our own reasonable expectations about how our system should respond. After we make our determination, we can proceed to the next test step or substep.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Creating Test Conditions



NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

A Basic Testing Template

- A basic test case template that can be used for either manual or automated testing.
- Each test step includes the action to be taken, the associated data, and the expected results.
- It's a good idea to name the test case with both mnemonic and numeric identifiers.
- The mnemonic name is a short description that conveys the essence of the test— for example, Stress, 1m Drop, or Network Performance.
- For the numeric identifier, you can use a Dewey
- decimal–style notation: for example, a test case that is used in the fifth test

	A	B	C	D	E
1	Test Case Name:	Mnemonic identifier			
2	Test ID:	Five-digit ID, XX.YYY: XX suite number, YYY test number.			
3	Test Suite(s):	The name of the test suite(s) that use this test case.			
4	Priority:	From quality risk coverage analysis			
5	Hardware Required:	List hardware in rows			
6	Software Required:	List software in rows			
7	Duration:	Elapsed clock time			
8	Effort:	Person-hours			
9	Setup:	List steps needed to set up the test			
10	Teardown:	List steps needed to return SUT to pretest state			
11					
12	**ID**	**Test Step/Substep**	**Result**	**Bug ID**	**Bug RP**
13	1.000	Major step			
14	1.001	Minor step (substep)			
15	1.002	Minor step (substep)			
16	2.000	Major step			
17					
18					
19	**Execution Summary**	**Status**			
20		**System Config ID**			
21		**Tester**			
22		**Date Completed**			
23		**Effort**			
24		**Duration**			

A Basic Testing Template

- The next entry in the header lists the name of the test suite (or suites) in which the test case will be used. Because you might use a given test case in multiple suites, this entry could potentially be large.
- However, in practice most test cases are used in only one test suite, so including the name of the suite provides some useful information with only infrequent confusion.
- To achieve the test sequencing benefits of risk-based testing, each test case should inherit the risk priority number of its parent risk item.
- In some cases, you might decide to modify these priorities, and, if so, it is recommended documenting that decision.
- The next entries in the header address resource requirements. For two of these entries, we list, row by row, the hardware and software needed to run the test.
- You might want to restrict these lists to the non-obvious. *For example, if you are testing a Windows-based application and the standard test environment includes the most recent version of Microsoft Windows, Microsoft Office, Trend Micro AntiVirus, and GoToMyPC, you needn't duplicate that listing for every test case.*
- The entries for Duration and Effort specify how long it will take to run the test, in clock time and in person-hours, respectively.

A Basic Testing Template

- In creating these estimates, you have two alternatives. You can assume that the test will pass.
- In www.it-this case, you will need to include additional time for expected bug finding and reporting in your estimation process.
- Alternatively, you can build expected bug finding and reporting time estimates into each test case, which allows you to estimate test passes based on the total test case estimation, plus additional time for confirmation testing and reactive testing.
- Based on the relationship between duration and effort, you can infer the human resources needed to run the test.

For example, do you need two people to run the test, one in front of each terminal? You would if you were testing the multiplayer features in video game. Do you need one person for just an hour, even though the test runs for 12 hours? You might if you were describing an automated test that runs unattended overnight.

- In the final two entries of the header, you need to specify the setup procedures and the teardown procedures.
- Sometimes there are none, but typically you'll need two or three steps at the beginning and end of a test, such as installing or uninstalling an application.

For example, when the tester concludes a test, she might need to delete sample files created during the test in order to return the system under test to its original state. With the preliminaries out of the way, let's move on to the test case proper.

A Basic Testing Template

- A test case is fundamentally a sequence of actions, performed serially, in parallel, or in some combination, that creates the desired test conditions.
- The template breaks down these actions into steps and substeps.
- Each step or substep might involve the use of special test data, either entered as part of running the test step or prior to starting the test case during the setup steps.
- During test design, the test engineer determines the expected results of each step or substep, which might be in an output, a system state, a timing or sequencing result, or some other observable behavior.
- Alternatively, the test engineer might determine how the test system should determine the expected result; e.g., the test system might use a legacy system or other program to generate the expected result during test execution.
- Each step or substep has a numeric identifier.
- Again, using a Dewey decimal–style notation, you can number steps sequentially starting from 1 and then number their subordinate substeps decimally (1.001, 1.002, and so forth).
- This numbering allows you to refer to specific test steps by combining the test case's numeric identifier (from the header) with the test step identifier— for example, the second substep of step 5 in test case 9.010 can be unambiguously identified as 9.010.5.002. This method is useful in bug reports and in discussions with testers

A Basic Testing Template

- Following the execution of a test case, one of three statements will typically hold true for each step or substep:
- The tester ran the test step or substep and observed the expected result, the whole expected result, and nothing but the expected result.
- The test step or substep did not locate any bugs.
- The tester should record Pass in the Result column.
- The tester ran the test step or substep, and the outcome was, to a greater or lesser extent, unexpected.
- The test case successfully identified some untoward behavior that the tester can now report in the bug-tracking database.

How to classify the test case?

- If the unanticipated result was something along the lines of a CPU catching fire, a program crashing, or a system lockup, the tester should enter Fail in the Result column.

A Basic Testing Template

However, what if the unexpected result doesn't affect the correct operation of the functionality under test?

- Development might see your team as unfair and alarmist if the tester classifies this test as Fail. It is important, however, to keep track of the test case that did find a new bug, so we don't want your testers to record it as a Pass.
- For these situations, you can use the word Warn as the documented result.
- A Warn entry can also cover some of the gray areas between complete failure and tangential failure— for example, if system under test performs the functionality under test correctly but
- also displays an incorrectly spelled help message. The tester did not run the test step or substep.
- If the tester could not run the step or substep— for example, because a known bug prevented the specified action or because the test environment lacks necessary resources— the tester should record Block in the Result column.
- If the tester chose not to run the test step or substep, the test should mark the step or substep as Skip. In either case, the tester should explain this omission.

A Basic Testing Template

- Status entry Pass, Warn, or Fail, depending on the results of the test case.
- The tester might also record Block or Skip if applicable.

- Since test cases consist of multiple steps, you might find that you need a hierarchical rule for assigning test case status based on test step status, such as the following:

1. If any test step or substep is in progress, then record the entire test case as IP.
2. Else, if any step or substep is blocked, then record the entire test case as Block.
3. Else, if any step or substep failed, then record the entire case as Fail.
4. Else, if any step or substep warned, then record the entire case as Warn.
5. Otherwise, all the steps must have passed, so record the entire case as Pass.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

A Stress Test Case for DataRocket

	A	B	C	D	E
1	Test Case Name:	CPU and Memory			
2	Test ID:	2.001			
3	Test Suite:	Load, Capacity and Volume			
4	Priority:	High			
5	Hardware Required:	One Datarocket Service			
6	Software Required:	Load, Capacity and Volume Test Suite			
7		Windows Server 2008			
8		Solaris			
9		RedHat Linux			
10	Duration:	4			
11	Effort:	4			
12	Setup:	Install Windows Server 2008			
13		Install Linux			
14		Install Solaris			
15	Teardown:	Burn drive images of each OS installation.			
16	**ID**	**Test Step/Substep**	**Result**	**Bug ID**	**Bug RPN**
17	1.000	Test CPU load on Windows Server 2008.			
18	1.001	Install Windows Servcer 2008	Pass		
19	1.002	Install LCV Test Suite.	Warn	760	6
20	1.003	Run LCV Test Suite, CPU Module, for one hour. Check log file for failures on exit.	Pass		
21	2.000	Repeat steps 1.001-1.003 for Solaris.	Pass		
22	3.000	Repeat steps 1.001-1.003 for Linux.	Fail	762	1
23	**Overall**	**Status**	Fail		
24		**System Config**	B,A,C		
25		**Tester**	LTW		
26		**Date Completed**	7/12		
27		**Effort**	5.5		
28		**Duration**	5.5		

The basic test case template used for a DataRocket stress test

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Another Useful Test Case Template

	A	B
1	Test Case Name:	Mnemonic identifier
2	Test ID:	Five-digit ID, XX.YYY: XX suite number, YYY test number.
3	Test Suite(s):	The name of the test suite(s) that use this test case.
4	Priority:	From quality risk coverage analysis
5	Hardware Required:	List hardware in rows
6	Software Required:	List software in rows
7	Duration:	Elapsed clock time
8	Effort:	Person-hours
9	Setup:	List steps needed to set up the test
10	Teardown:	List steps needed to return SUT to pretest state
11		
12	**Screen/Field**	**Test Step/Substep**
13	Screen1	Screen name
14	Field1	Input value for field1.
15	Field2	Input value for field2.
16	Screen2	Screen name.
17	Expected Results	The result at the final (test condition) input.
18		
19	**Execution Summary**	
20	**Status**	
21	**Bug ID**	
22	**System Config ID**	
23	**Tester**	
24	**Date Completed**	
25	**Effort**	
26	**Duration**	

A screen- and field-oriented test case template

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

IEEE 829 Test System Templates

	A	B	C	D
1	Test Case Name:	Configure/Register Windows	Configure/Register Solaris	Configure/Register Linux
2	Test ID:	3.001	3.002	3.003
3	Test Suite(s):	Basic Functionality	Basic Functionality	Basic Functionality
4	Priority:	High	High	High
5	Hardware Required:	One DR Server	One DR Server	One DR Server
6	Software Required:	Windows Server 2008	Solaris	Linux
7	Duration:	4	4	4
8	Effort:	2	2	2
9	Setup:	Install blank HD.	Install blank HD.	Install blank HD.
10	Teardown:	Save "Clean" WS2008 drive.	Save "clean" Solaris drive.	Save "clean" Linux drive.
11				
12	**Screen/Field**	**Test Step/Substep**	**Test Step/Substep**	**Test Step/Substep**
13	Welcome Screen			
14	Welcome Message	Check against docs.	Check against docs.	Check against docs.
15	Operator Name	Enter first, MI, last.	Enter first, MI, last.	Enter first, MI, last.
16	Company Name	Enter Name (min/max).	Enter name (min/max).	Enter name (min/max).
17	Company Address	Enter various options.	Enter various options.	Enter various options.
18	Next Button	Mouse or tab/enter.	Mouse or tab/enter.	Mouse or tab/enter.
19	Install Screen			
20	Select OS	Windows Server 2008	Solaris	Linux
21	Enter Disk Registration	Try invalid, then valid.	Try invalid, then valid.	Try invalid, then valid.
22	Confirm CD-ROM Insert	Try no, see CD request, then try yes w/o CD, see error message, then try yes w/CD.	Try no, see CD request, then try yes w/o CD, see error message, then try yes w/CD.	Try no, see CD request, then try yes w/o CD, see error message, then try yes w/CD.
23	Confirm Install Start	Confirm no start if 'YES' not entered exactly as spelled.	Confirm no start if 'YES' not entered exactly as spelled.	Confirm no start if 'YES' not entered exactly as spelled.
24	Complete Screen			
25	Success Message	Check against docs.	Check against docs.	Check against docs.
26		Reboot; verify WS2008 load. Verify automatic call to reg server.	Reboot; verify Solaris load. Verify automatic call to reg server.	Reboot; verify Linux load. Verify automatic call to reg server.
27	*Repeat above using a bad disk.*	Verify failed install, "Fail" message, no registration.	Verify failed install, "Fail" message, no registration.	Verify failed install, "Fail" message, no registration.
28				
29	**Execution Summary**			
30	**Status**	Warn	Fail	Pass
31	**Bug ID(s)**	772	774	
32	**System Config ID**	B	A	C
33	**Tester**	LTW	HS	LTW
34	**Date Completed**	7/19	7/20	7/21
35	**Effort**	3	5	2
36	**Duration**	6	7	4

Figure 3-8 A screen-oriented registration test case

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

IEEE 829 Test Design Templates

	A	B	C	D	E
1	**1.0**	**Introduction**			
2	1.1	Document Identifier			
3	1.2	Scope			
4	1.3	References			
5					
6	**2.0**	**Details of the Level Test Design**			
7	2.1	Features to be tested			
8	2.2	Approach refinements			
9	2.3	Test Identification			
10	2.4	Feature pass/fail criteria			
11	2.5	Test deliverables			
12					
13	**3.0**	**General**			
14	3.1	Glossary			
15	3.2	Document change procedures and history			

The IEEE 829 test design template

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

IEEE 829 Test Case Templates

	A	B	C	D	E	F
1	**1.0**	**Introduction (once per document)**				
2	1.1	Document Identifier				
3	1.2	Scope				
4	1.3	References				
5	1.4	Context				
6	1.5	Notation for description				
7						
8	**2.0**	**Details (once per test case)**				
9	2.1	Test case identifier				
10	2.2	Objective				
11	2.3	Inputs				
12	2.4	Outcome(s)				
13	2.5	Environmental needs				
14	2.6	Special procedural requirements				
15	2.7	Intercase dependencies				
16						
17	**3.0**	**Global (once per document)**				
18	3.1	Glossary				
19	3.2	Document change procedures and history				

IEEE 829 test case template

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

IEEE 829 Test Procedure Templates

	A	B	C	D	E	F	G	H	I
1	**1.0**	**Introduction**							
2	1.1	Document Identifier							
3	1.2	Scope							
4	1.3	References							
5	1.4	Relationship to other procedures							
6									
7	**2.0**	**Details**							
8	2.1	Inputs, outputs, and sp[No Title]nts							
9	2.2	Ordered description of the steps to be taken to execute the test cases							
10									
11	**3.0**	**General**							
12	3.1	Glossary							
13	3.2	Document change procedures and history							

IEEE 829 test procedure template

On Test Design and the Smart Use of Templates

- If you are persistent, you might ask, of course, ‘‘Fine, wise guy, but how did the test engineers do that?’’

In general, there are five main categories of test design techniques:

- **Behavioral tests of functionality:** Those tests that focus on what the system does, the real-world problems it solves. Such tests check the accuracy, suitability, interoperability, security, and compliance with applicable standards of the functional elements of the system.
- **Behavioral tests of non-functional elements:** Those tests that focus on how the system does what it does, the behaviors it displays while it solves problems. Such tests check usability, reliability, resource utilization, performance, maintainability, and portability of the system.
- **Structural tests:** Those tests based on internal design and implementation of the system. Such tests look at control flows, data flows, architecture, and the like.
- **Experience-based tests:** Those tests based on the experience and skills that testers have gained from testing similar systems. Similar in this case can mean similar domain, similar technology, or similar types of tests.
- **Defect-based tests:** Those tests based on defects typically found in similar systems. Similar in this case again can mean similar domain, similar technology, or similar types of tests.

How Detailed? The Effects of Precision

- An important decision when thinking about documenting test cases is how much detail the test engineers should include. How precisely should the test engineers document the actions, data, and expected results?
- The decision involves a set of trade-offs.
- On the one hand, extensive, detailed, precise test documentation assumes less knowledge on the part of the testers, which often allows you to use testers with lower levels of skill than you might otherwise require.
- It also supports reproducibility: if nothing is left to the judgment of an individual tester, you usually see less variability in two evaluations of the same test by two different people, or even by the same person at different times.
- In addition, if development is involved in the review process for test cases as your team constructs them, spelling out the details serves to communicate, clearly and unambiguously, what types of failures we're looking for and what behaviors we'll consider buggy.
- On the other hand, writing precise, detailed, unambiguous test documentation can involve considerably more work than writing a laconic one.

How Detailed? The Effects of Precision

- The difference in effort involved in creating a concrete set of test specifications (those that leave little to the discretion of the tester) or a logical set of test specifications (those that require tester skill, experience, and judgment) is about ten times the effort.
- Obviously, you have to write more, but you also must unlearn information that seems obvious to you and your test engineers but could mystify or confuse the test technicians and any others who might
- become involved in testing.
- If you've ever tried to explain what testing is all about to someone with no experience in the field, you already know that this can be quite difficult.
- Understanding the level of detail required for the test team you have assembled (or intend to assemble) is, you find, the key to writing effect test cases.
- The logical/concrete test case continuum

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

The logical/concrete test case continuum

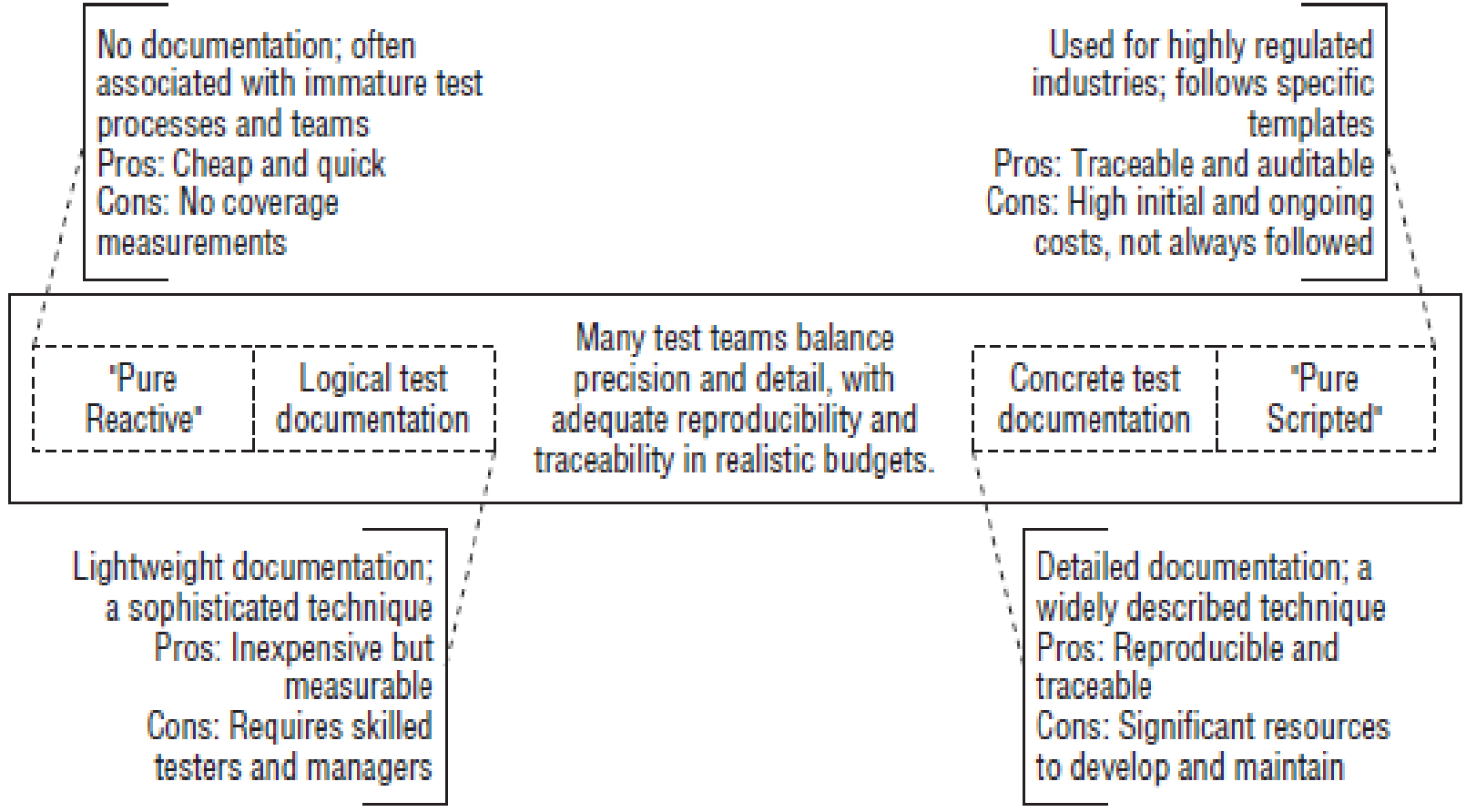


Figure 3-12 The logical/concrete test case continuum

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Avoiding the Dreaded Test Escape: Coverage and Regression-Test Gaps

- A common metric of test team effectiveness measures whether the test team manages to find a sizeable majority of the bugs prior to release.
- The production or customer bugs are sometimes called test escapes.
- The implication is that your test team missed these problems but could reasonably have detected them during test execution.
- You can quantify this metric as follows:

Defect Detection Effectiveness = bugs (test) bugs (test) + bugs (production)

- Note the word reasonably. You are probably not going to score 100% on this metric consistently.
- If your testers could have found the bug only through unusual and complicated hardware configurations or obscure operations, that bug should not count as a test escape.
- Moreover, the count of the bugs found in the field should not include bugs previously detected by testing; in other words, if the testers found the bug during testing, but project management deferred fixing the bug, that bug should count as a bug found by test.
- Finally, note that in many environments one can assume that most field bugs are found in the first three months to a year after deployment, so you can estimate the defect detection effectiveness after that period.

Test Gaps

- Test escapes usually arise through one or a combination of the following types of problems:
- **A low-fidelity test system.** While a low-fidelity test system might cover a significant chunk of the product's features and operations, it doesn't cover the ones most important to your customers, usually as a result of poor engineering.
- **A regression-test gap.** The test suite does contain test cases covering the operation in which the bug surfaced, but the team ran the test cases before the bug was introduced in the system under test. Regression-test gaps arise from schedule or resource limitations or from planning errors.
- **A test-result interpretation error.** A tester fails to detect or report incorrect system behavior, assigns an excessively low priority or severity to the bug, or otherwise understates the significance of the problem.

Bad Coverage Decisions with the Best Intentions

- Coverage mistakes are easy to make, even if you spend time thinking about the issue.

Scenario: *Some years ago, I took over as a test manager in a company that had developed a neat way of automating testing, using a portable scripting language and the command-line functionality of the system under test. For two years my team and I worked with that test system, polishing it to a bright luster. We wrote an intelligent output comparison program. We had clean, maintainable test tools, under strict revision control. The system was robust, reliable, and fast. Furthermore, we analyzed test coverage in terms of the crucial databases and operating systems supported by the system under test, and we had most of them nailed. Nevertheless, we had significant test escapes, and the customers were unhappy. Accordingly, my managers were unhappy, and they made my team and me very unhappy by liquidating the test organization.*

- What happened? *I let the test tool determine my decisions about test coverage. As the saying goes, ''Give a child a hammer and the whole world becomes a nail.'' We had only two test suites that evaluated the user interface. We barely tested the utilities included with the package. We spent insufficient time understanding the unique uses and the special boundary conditions that plagued our customers. I took a couple of lessons away from this experience. The most important was to be sure that I decided on test coverage based on the quality risks that matter to my customers, my end users, and my colleagues in such areas as marketing, technical support, sales, business analysis, the help desks, and any others who are close to the users.*

Are You Testing What Development Is Building?

- If you are lucky enough to get written requirements and design specifications for your product, you probably already use them as a basis for test planning and test system development.
- You should also close the loop and ascertain the coverage.
- You can use a numeric approach, such as the one outlined for quality risk coverage in the following section, or you can use a cross-reference approach.
- To create a cross-reference document, carefully go through the requirements or specifications, and note (in a soft copy) the specific test cases that verify each requirement or specification.
- For those requirements not paired with a test case, decide whether there's a good reason to skip testing in that area.
- If there isn't, you have found a coverage problem you need to address.
- Apply this approach iteratively until you are happy with your test system's coverage.
- You can also circulate this marked-up cross-reference to your peers, especially key testing stakeholders, to be sure they're comfortable with what you plan to test.
- You can apply this approach even if you don't have requirements or design specifications documents.
- By listing the functional areas of the product at a general level and then subdividing them into more specific areas, you have a black-box decomposition of what the system is supposed to do.

Are You Testing What Development Is Building?

- Then proceed as just described, using either a numeric or a cross-reference approach.
- Iterate until you're satisfied, and circulate the document for review.
- Remember two caveats about using such a functional coverage approach, however.
- First, it measures testing of what the system does.
- Don't forget about what it doesn't do, shouldn't do, and should do — these factors are equally important to your customers.
- You'll want to add coverage analysis based on the risks you identified to check these areas, as described in the following section.
- Second, if you are focused narrowly on functions, you can easily overlook factors such as stability, performance, data quality, error handling and recovery, and other such system problems.
- A mistake in the cautionary tale related earlier was, at least in part, that of depending solely on a narrow functional coverage analysis to validate my testing approach.
- As your coverage analyses move further away from customer requirements and more into the minutiae of system functionality, you need to augment your approach by looking at what the customers care about.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Relating Quality Risks to Test Cases

Table 3-1 Relative Test-Coverage Analysis

VALUE	COVERAGE MEANING
0 (or blank)	The test case does nothing to address the quality risk.
1	The test case provides some level of indirect coverage for the quality risk.
2	The test case provides direct and significant coverage for the quality risk.

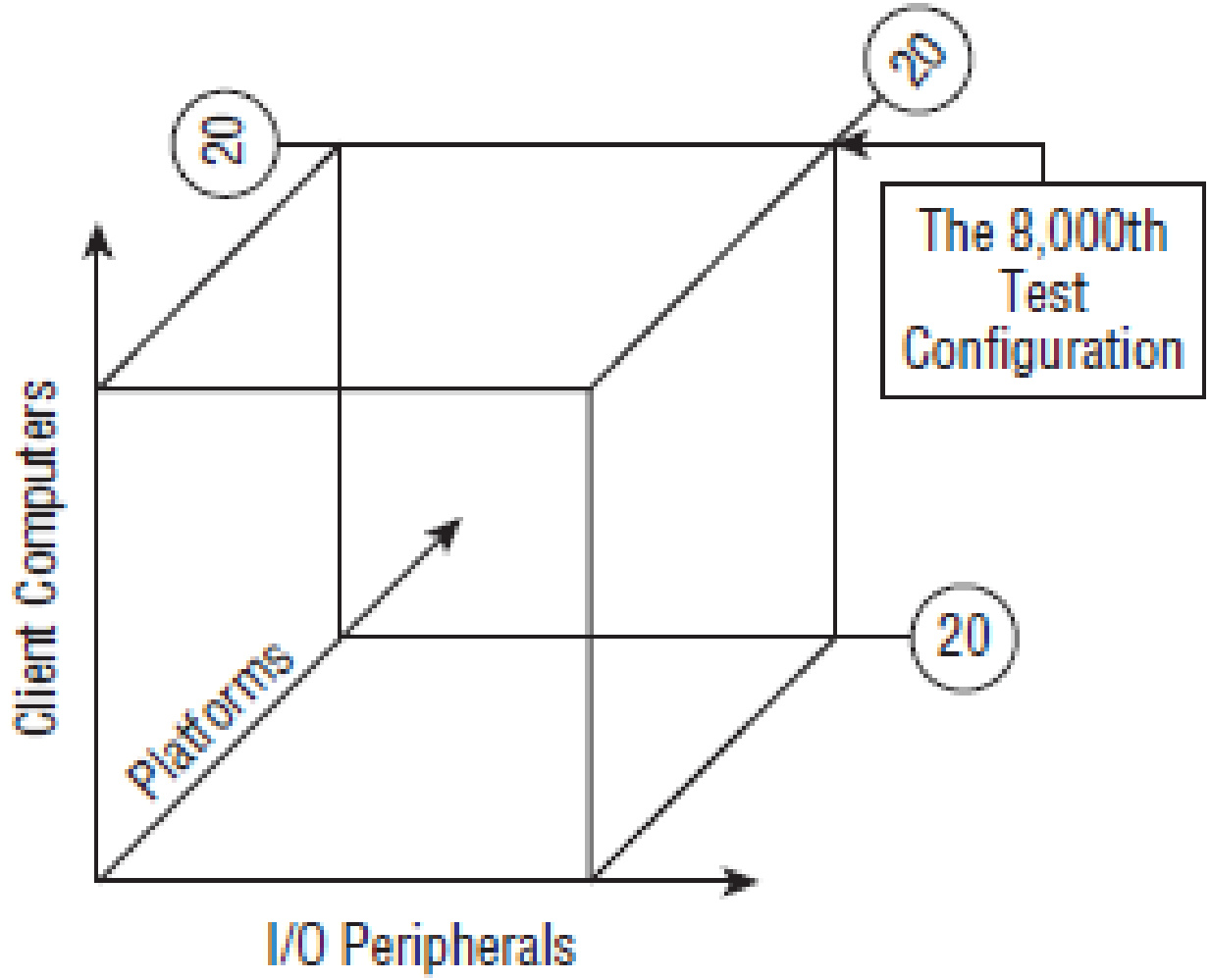
Configuration Coverage

- Let's suppose that you are testing SpeedyWriter, and your manager tells you to run only the installation and file-open tests. You not only will be allowed to retain your current level of staff and resources, but also will be given even more resources— an unlimited hardware budget.
- In return, all you have to do is test SpeedyWriter against every possible combination of client computer, platform (Web browser, operating system, and network), and I/O peripheral.

Would you do it?

- *For purposes of illustration, let's assume 20 possible variations of each variable: 20 different client computers; 20 operating system, browser, and network combinations; and, 20 combinations of printers, mice, keyboards, scanners, and video controllers. You would have to test 8,000 different configurations, give or take a few hundred impossible configurations, such as the use of a particular peripheral with an operating system that doesn't support it. If the two tests (installation and file-open) take about an hour, including setup and teardown time, you have about 2.75 person-years of work ahead of you. In addition to the time, imagine the expense. This simple example could easily cost $100,000 in hardware, operating systems, and server software alone.*

An intractable configuration coverage problem



An intractable configuration coverage problem

Bug Coverage

- To find bugs — the real question is, ‘‘Have we found enough bugs?’’
- Important bugs could remain because we haven’t tested enough.
- Unfortunately, we can’t know what we don’t know.
- If it were possible to know all the bugs in advance, the entire test effort would be unnecessary.
- Suppose, though, that you could estimate the total number of bugs in the system under test. Or, perhaps you could measure the bug-finding effectiveness of your test system.
- There are three techniques for solving these problems.
- Commonly used technique for estimating bug numbers relies on historical data.
- Based on the size of the current project, the sizes of past projects, and the number of bugs found in past projects, the test manager can estimate the number of bugs in the current project.

Suppose the current project is estimated at 5,000 lines of code. Further, suppose that historical data indicates one bug found in system test or, failing that, in the first six months following release for every 100 lines of code. In this case, we can estimate that either we find 50 bugs during system test or resign ourselves to the remainder surfacing after release.

Bug Coverage

- Line-of-code estimates being notoriously unreliable early in the project, though, this estimate is subject to considerable error.
- Alternatively, we can measure project size in function points.
- Function points are a software-sizing technique that assigns a given number of points to certain activities that the developers must build into a program, based on a detailed requirements specification.
- Users of this technique claim it is more reliable as an estimating tool.
- Suppose we expect— based on the requirements— that we'll implement 100 function points. Suppose further that we expect 1.5 bugs per function point, so 150 bugs would escape into the system test phase.
- We can either find about 150 bugs during system test, or deliver those bugs to the customers and users.
- The problem is that you'll need to have function point counts. Most projects and clients I've worked with don't have these.
- A more practical technique involves estimating the number of bugs based on development effort. *Suppose you know, from historical data, that for a given person-month of development effort, the programmers will introduce 30 bugs that will either surface during system test or after release.*

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Predicting bugs and bug find/resolve time

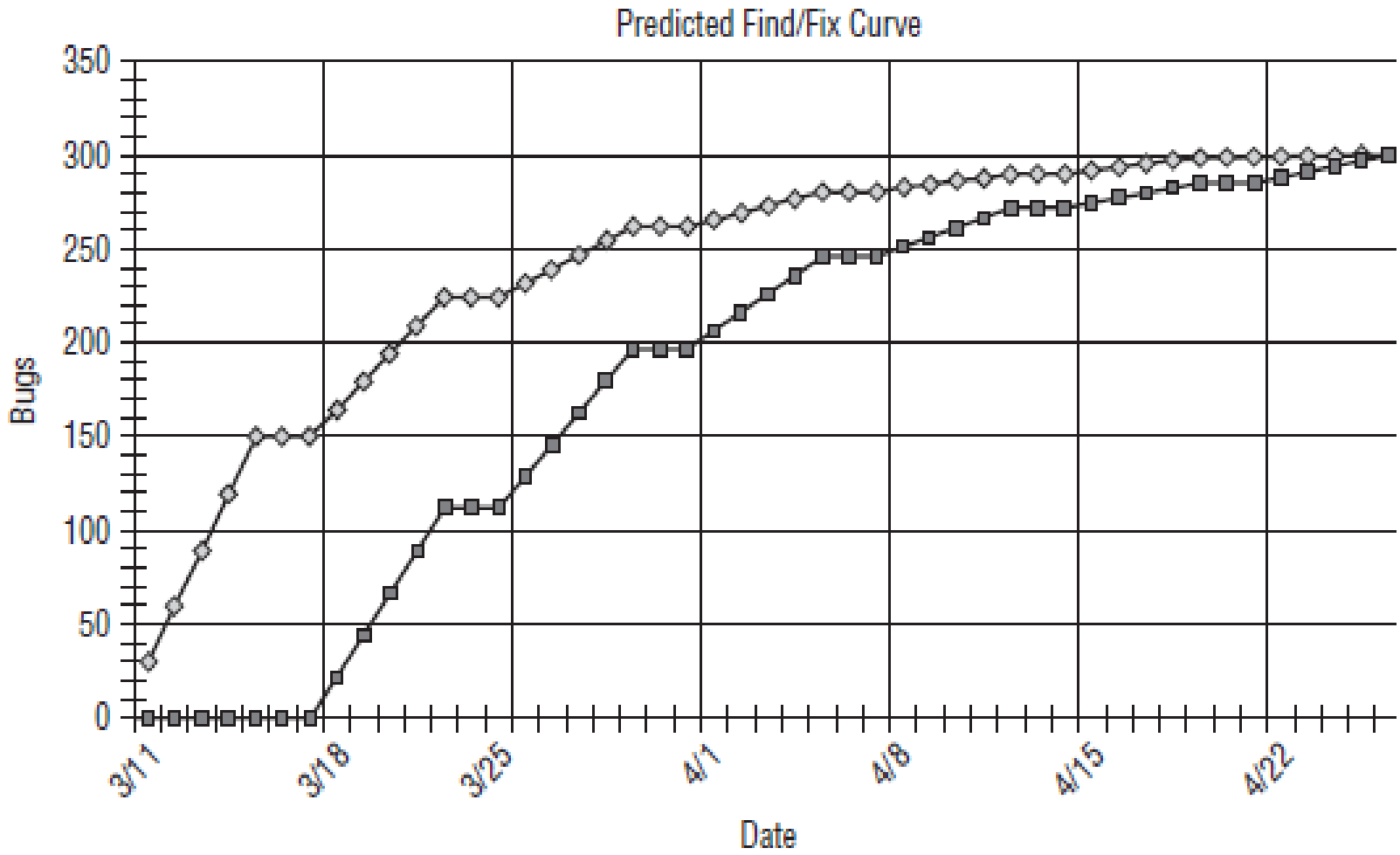


Figure 3-14 Predicting bugs and bug find/resolve time

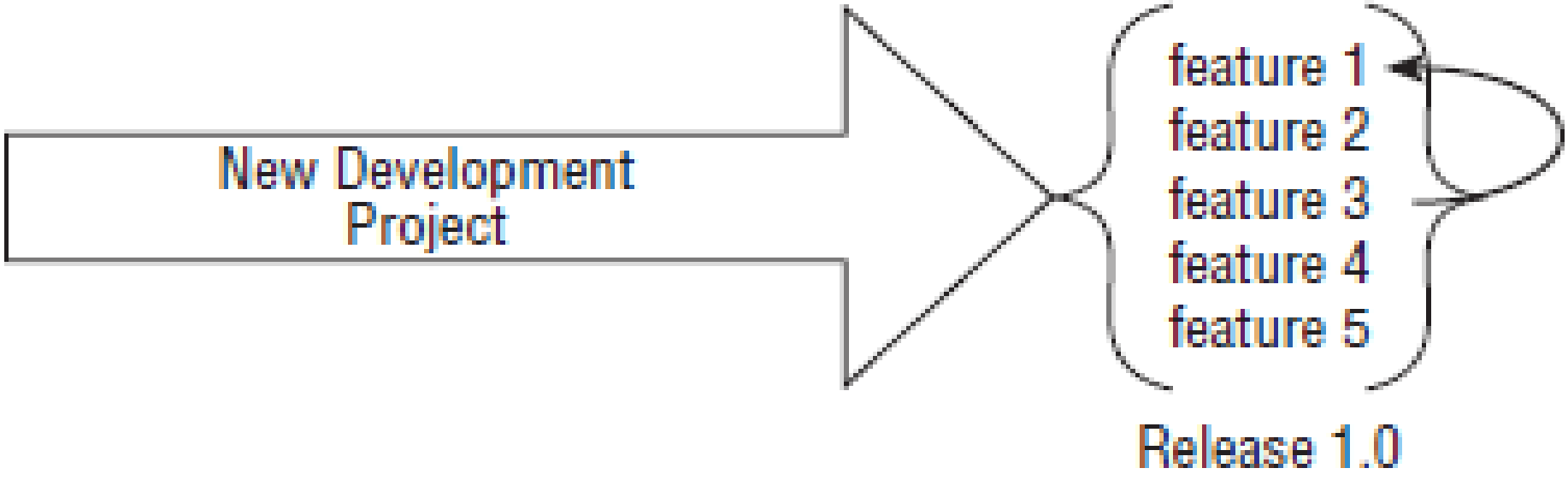
Regression-Test Gaps

- The concept of regression is straightforward but sometimes misunderstood.
- Formally, you can say that if, as a result of a change in the system under test, a new revision of the system, Sn+1 contains a bug not present in the previous revision, Sn, the quality of the system has regressed.
- Plainly speaking, regression occurs when some previously correct operation misbehaves.
- If a new revision contains a new piece of functionality that fails without affecting the rest of the system, that's not regression.
- Usually, you'll detect regression when test cases that passed previously now fail.
- Some regression, though, is so obvious that no test is required to find it— flaws in a system's case or a glitch in software installation, for example.
- Or, you can have Zen regression: if a bug crawls into the system but no one detects it, you will find this regression, like the sound of the unattended falling tree, hard to prove!

Regression-Test Gaps

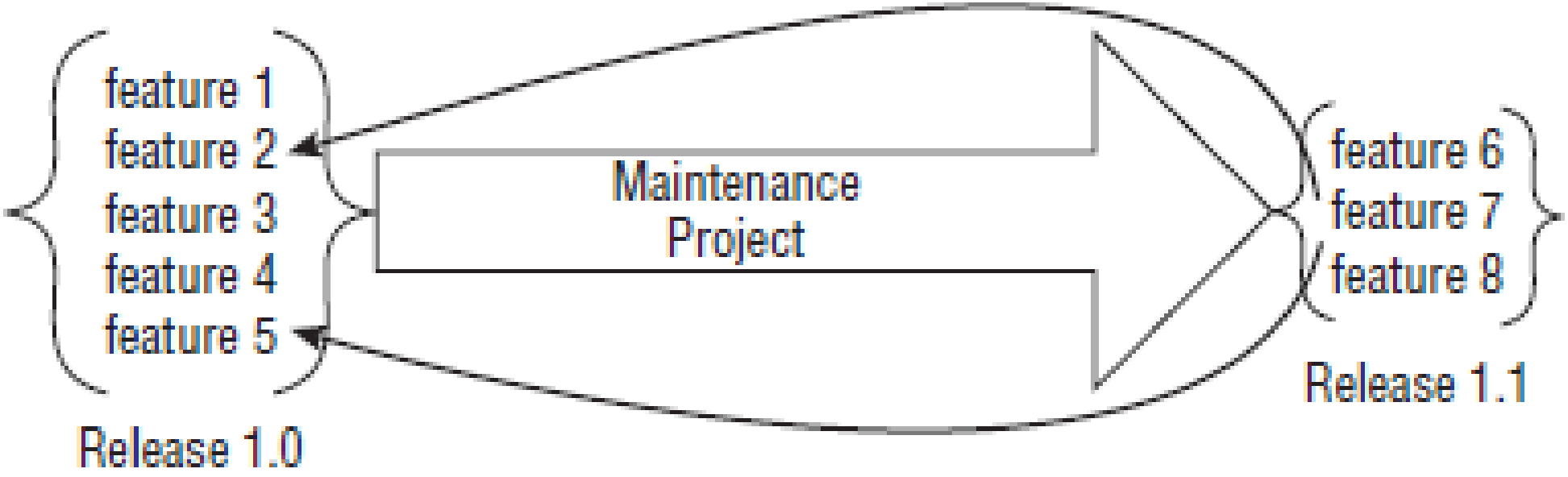
- There are two instructive ways to think about regression.
- The first has to do with the feature-space separation between the change introducing the
- regression bug and the regression failure associated with it.
- The most painless is the local regression, where a fix or change creates a regression failure in the same feature area where the programmer made the change.
- We call this painless because a confirmation test of the fix might well reveal the new failure.
- Similarly, when a change or fix exposes an existing bug, for example by making a particular area of the system accessible for the first time, that’s not too hard to detect, either.
- The tougher situation is when you’re dealing with a remote regression, where a fix in one area breaks something in another area.
- The classic example here is where a change affects the data stored by one feature, and this new format or type of data causes failure of another area of the system that uses the data.

Regression of a new feature



Regression of a new feature

Regression of existing features



Regression of existing features

Is Automation a Complete Solution?

- Some people suggest test automation as a solution to this problem, and it can be— at least partially.
- An automated test system runs more tests in a given period than a manual system does (at least if it works effectively and efficiently).
- Consequently, the regression test gap is reduced.
- As a test manager, you will receive lots of brochures touting (expensive) test automation tools, so bear the following in mind when you read them.
- For a stable system, you can develop automated tests and run them repeatedly, with some minor modifications to one or two test cases each time.
- However, when interfaces, major functions, or supported environments evolve and change rapidly, automated tests can require significant maintenance.
- It's great if you can anticipate how these changes will affect your automated regression tests.
- Sometimes you can.
- In my experience, though, most automated regression-test failures aren't failures at all, at least not in the sense of a failure as symptom of a bug.
- Rather, many of these mismatches between expected and actual results arise from unexpected, but correct, consequences of other changes.

Is Automation a Complete Solution?

- Try as you might, you usually cannot change the all of the expected results for the automated tests prior to starting test execution.
- So, you're often forced to wait until test execution to see the exact effects of the changes on the tests and their results.
- During the thick of test execution, though, you won't always have the time and resources to fix the tests.
- This might tempt you to wait until the product is released to maintain and/or develop automated tests for the functionality just developed, but systems in major development run the highest risk of regression, since the level of code change is high.
- This is just a cruel irony of life in a test organization.
- That said, the most practical approach is, in my experience, to focus most functional automation efforts on functionality introduced in the previous release, as these features will — we hope— tend to remain stable from one release to the next after initial release.
- This means that, often, we can apply automated testing only to features already in the production or released system.
- The new features will require manual regression testing prior to release, as the system changes

Four Ways to Spread Tests Across Cycles

- Test automation or no, the time will probably come when you will be forced to select only a subset of the test system for certain test cycles — or even test projects.
- When this happens, you can use these four alternative tactics to minimize regression-test gaps:
- Assigning an aggregate risk priority number to each test suite in advance, based on the risk priority number for each test case, and then running the test suites in a way that favors the higher-priority tests. We call this the static priority approach.
- Assigning the risk priority numbers to each test suite in advance, but adjusting them dynamically as each test cycle begins, and then running the test suites in priority order. I call this the dynamic priority approach.
- Randomly distributing the test suites across the test cycles. We call this the shotgunning approach.
- Running the entire set of test suites straight through as many times as possible (definitely more than once). We call this the railroading approach.

A Frenetic DataRocket System Test Schedule

Table 3-2 A Frenetic DataRocket System Test Schedule

CYCLE	START	END	MB REV	BIOS REL	% TESTS
1	7/1	7/7	A	X1	100
2	7/8	7/9	A	X2	50
3	7/10	7/11	A	X3	50
4	7/12	7/13	A	X4	50
5	7/14	7/15	B	X5	50
6	7/16	7/17	B	X6	50
7	7/18	7/19	B	X7	50
8	7/20	7/21	B	X8	50
9	7/22	7/23	B	X9	50
10	7/24	7/30	B	X10	100

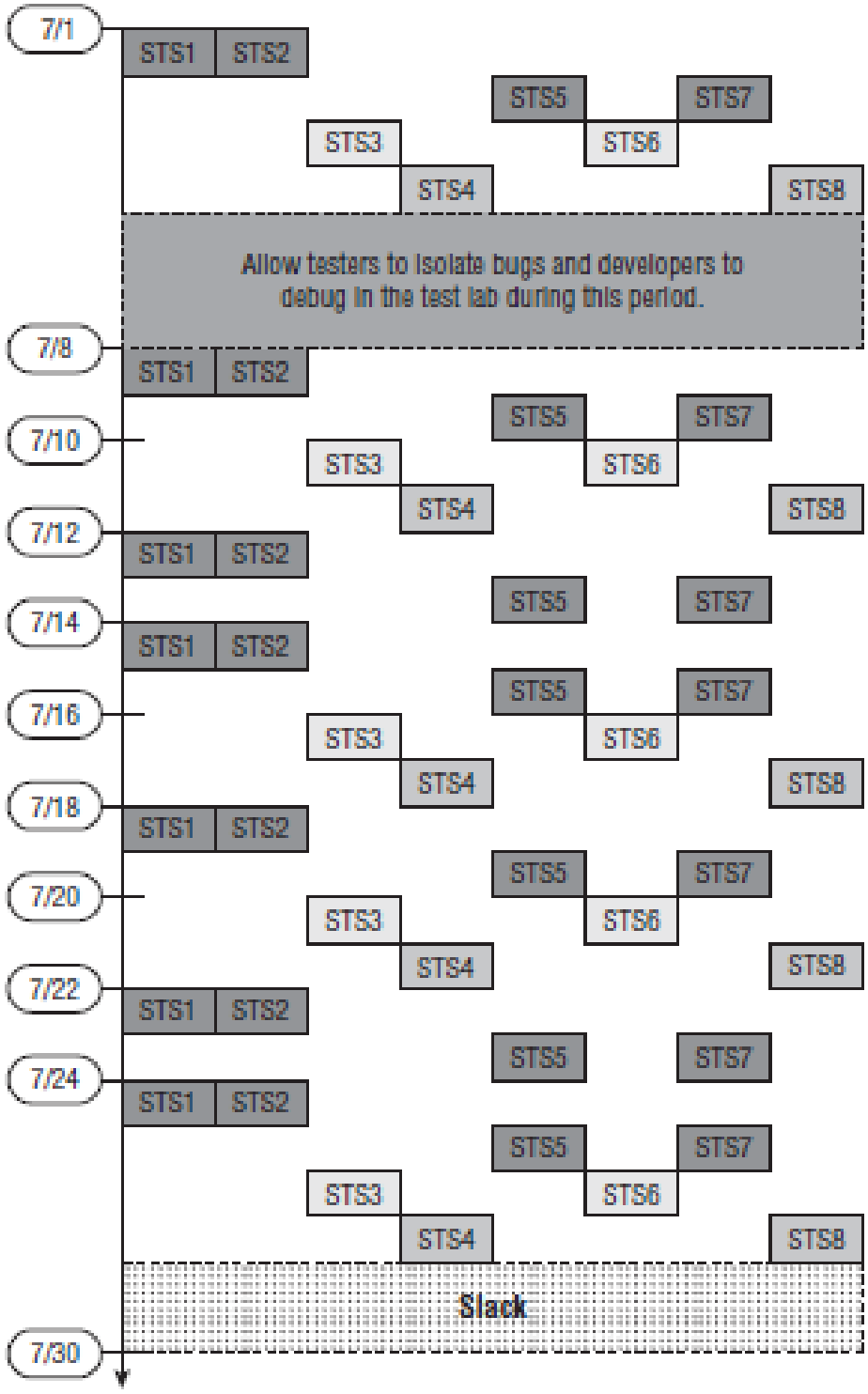
Static Priority

- Suppose we use an averaging technique across the test cases to determine the average risk priority for the test suite.
- Based on a three-level division of priority, let's suppose that the high-priority subset of the test system consists of test suites STS1, STS2, STS5, and STS7, and takes two days to run.
- The medium-priority subset containing STS3 and STS6 takes one day.
- The low-priority portion containing STS4 and STS8 also takes one day.
- The complete first pass provides a baseline for the rest of the testing and alerts development to every bug I can find in the system www.it-right from the start.
- The three-day hiatus from organized test suite execution allows testers to isolate all the bugs revealed by the first run of the test suites and gives the developers access to the test lab for debugging purposes

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informa

A road map for test selection based on static prioritization



17 A road map for test selection based on static prioritization

Dynamic Priority

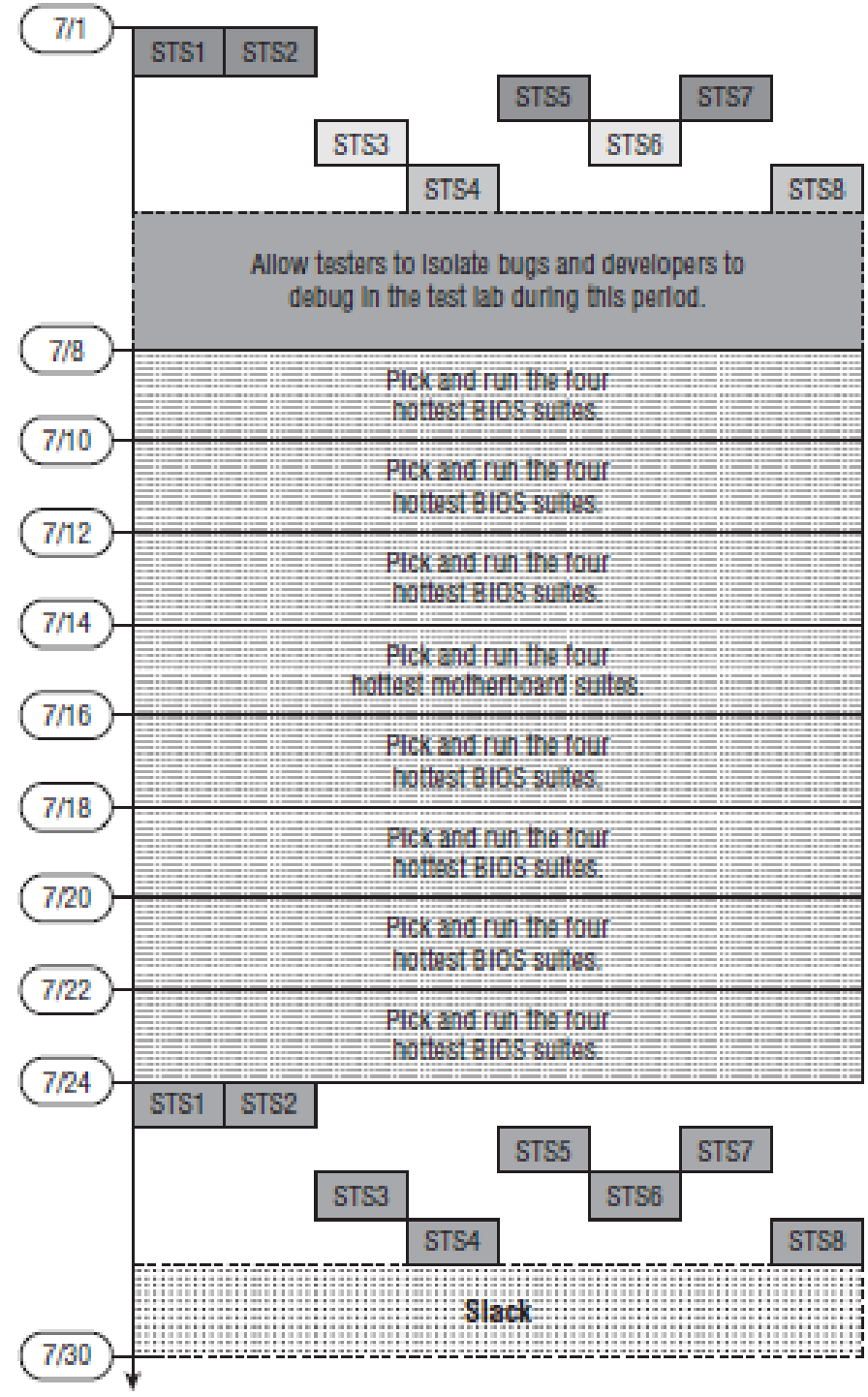
- The priority-order approach might have raised a question for you: how do you know in advance which test suites should have a high priority? Of course, the quality risk analysis is an educated guess, and you sure hope you got that right.
- However, the odds are pretty good that you will be proven partially wrong.

Suppose that you assign a high priority to DataRocket's thermal test (STS1) because an early engineering evaluation unit caught fire at one point. Also suppose that you assign a low priority to the USB, FireWire, serial, and parallel port test (STS4) because development has found no problems in this area. By the end of July 4, though, you discover that you have the priorities backward: the thermal test passes with headroom to spare, but the USB test is a target-rich environment. It turns out that the developers forgot to test the interaction of power management with USB hot plugging and hot unplugging, and the system crashes when you try this. Now, suddenly, STS4 is a high-priority suite, and STS1 is a low-priority suite. You can handle this situation by conceding in advance that the priorities you come up with ahead of time might change. With this concession, you simply plan to run four test suites, selected based on the priorities of the moment, during the second through ninth test cycles (with complete passes at the beginning and the end, as before).

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

A dynamically prioritized road map for test selection



A dynamically prioritized road map for test selection

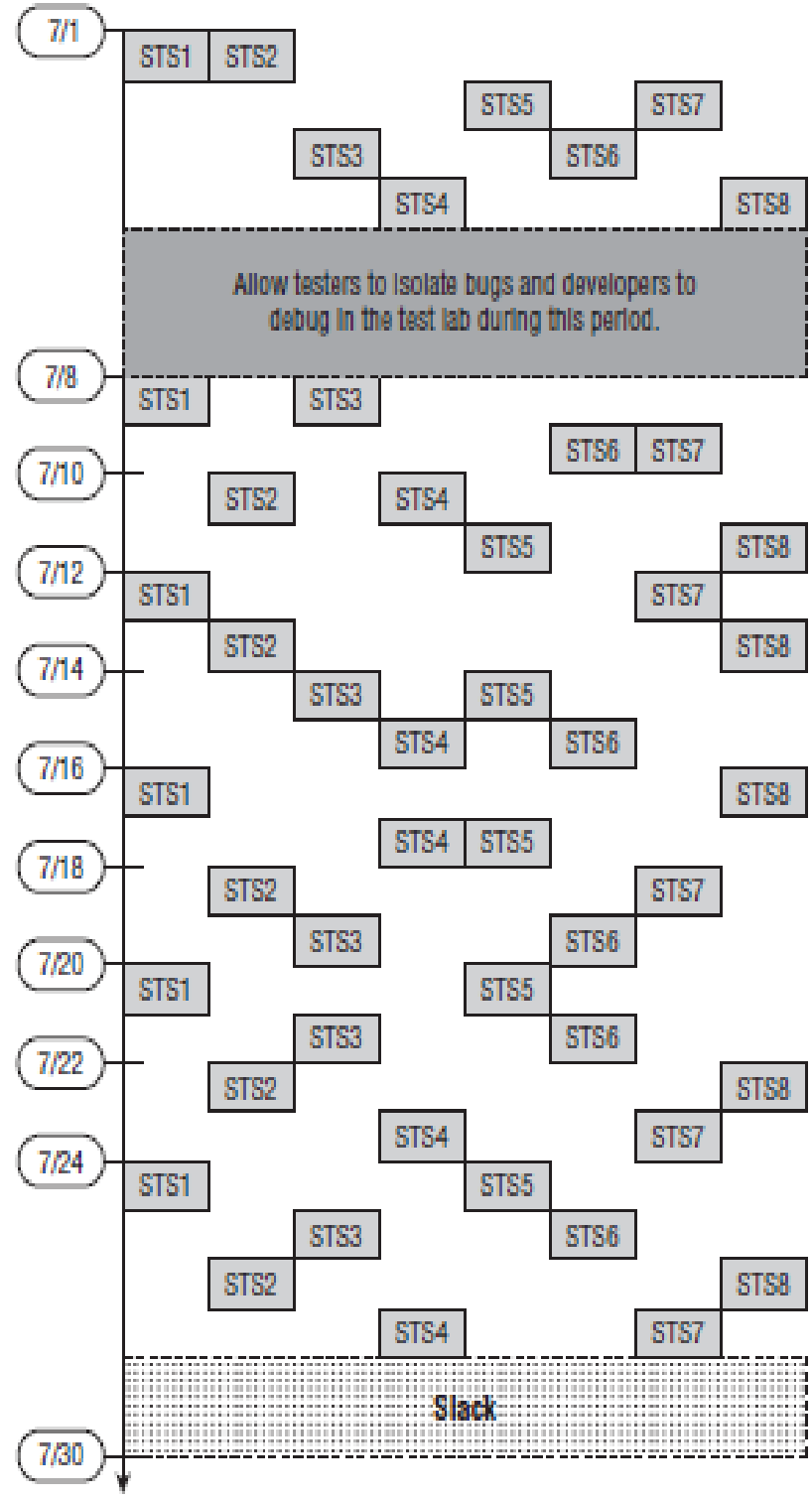
Shotgunning

Suppose that you want to have a plan in place to help impose some discipline, but you do not have sufficient data to finalize the sequencing of tests in advance. At the time of test planning, you are forced to consider all test suites equally important. So, you can distribute the suites randomly in test cycles 2 through 9. (Cycles 1 and 10 remain total coverage cycles as before.).This shotgun approach is more robust than the previous two approaches in the face of rapid, unpredictable change. Since you are decoupling your test suite selection from the specific test deliverables, a change in those deliverables doesn't affect your plan. What if the BIOS release doesn't show up on July 10? With the previous schedules, you might have wondered what to do next. With a shotgun road map, however, four test suites have not yet been run against the X2 BIOS release, so you can simply continue with your plan to run those tests.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

A road map for test selection based on shotgunning



19 A road map for test selection based on shotgunning

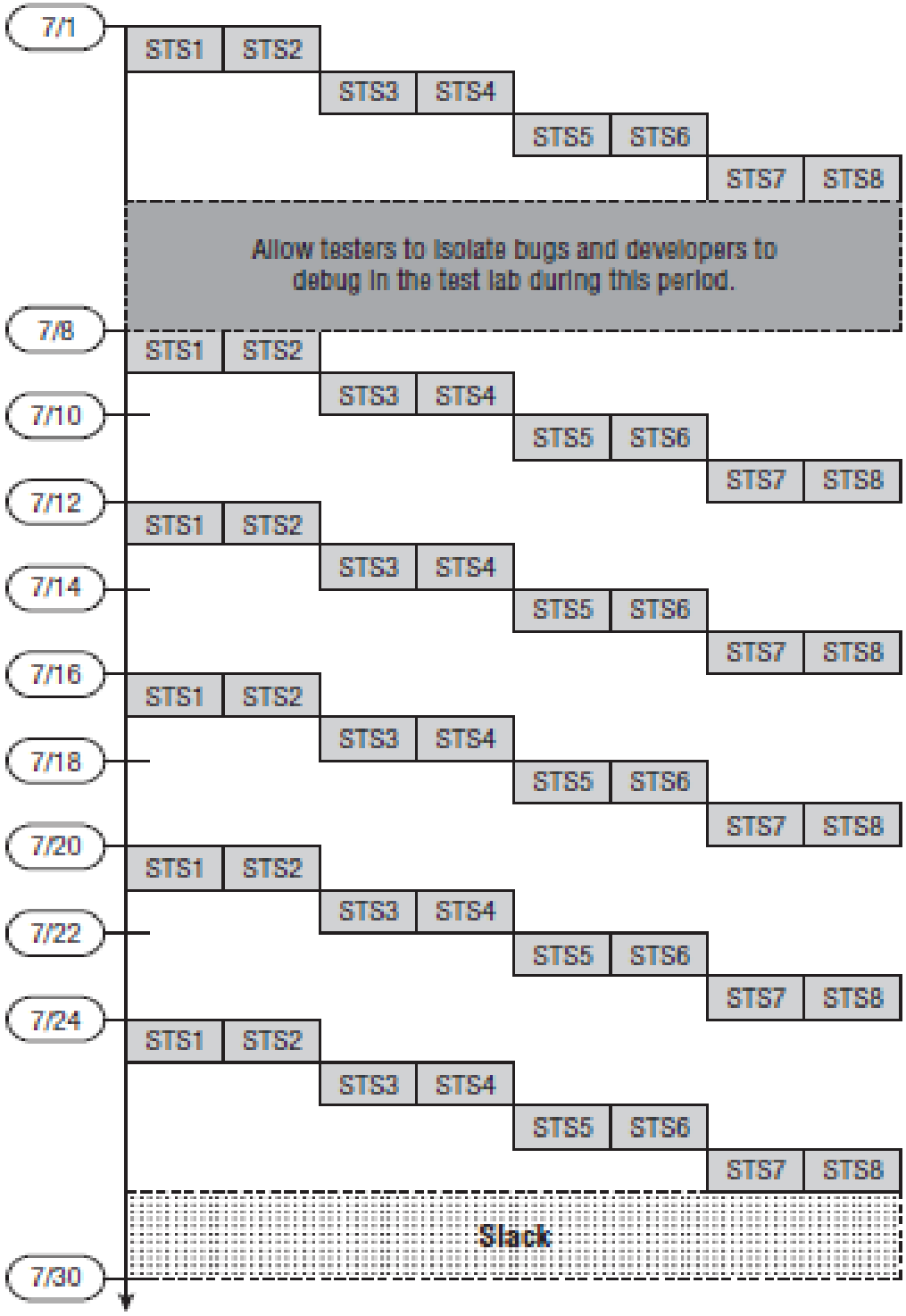
Railroading

The shotgun approach looks clever on paper, but the premise might strike you as flawed. After all, if the test suites are equally important, why go to the trouble of randomizing their run order? You could just as easily use the test suite numbering — STS1 through STS8— as the sequence. In this case, the testing rolls on regardless of what's going on in the project.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Infor

A road map for test selection based on railroading



-20 A road map for test selection based on railroading

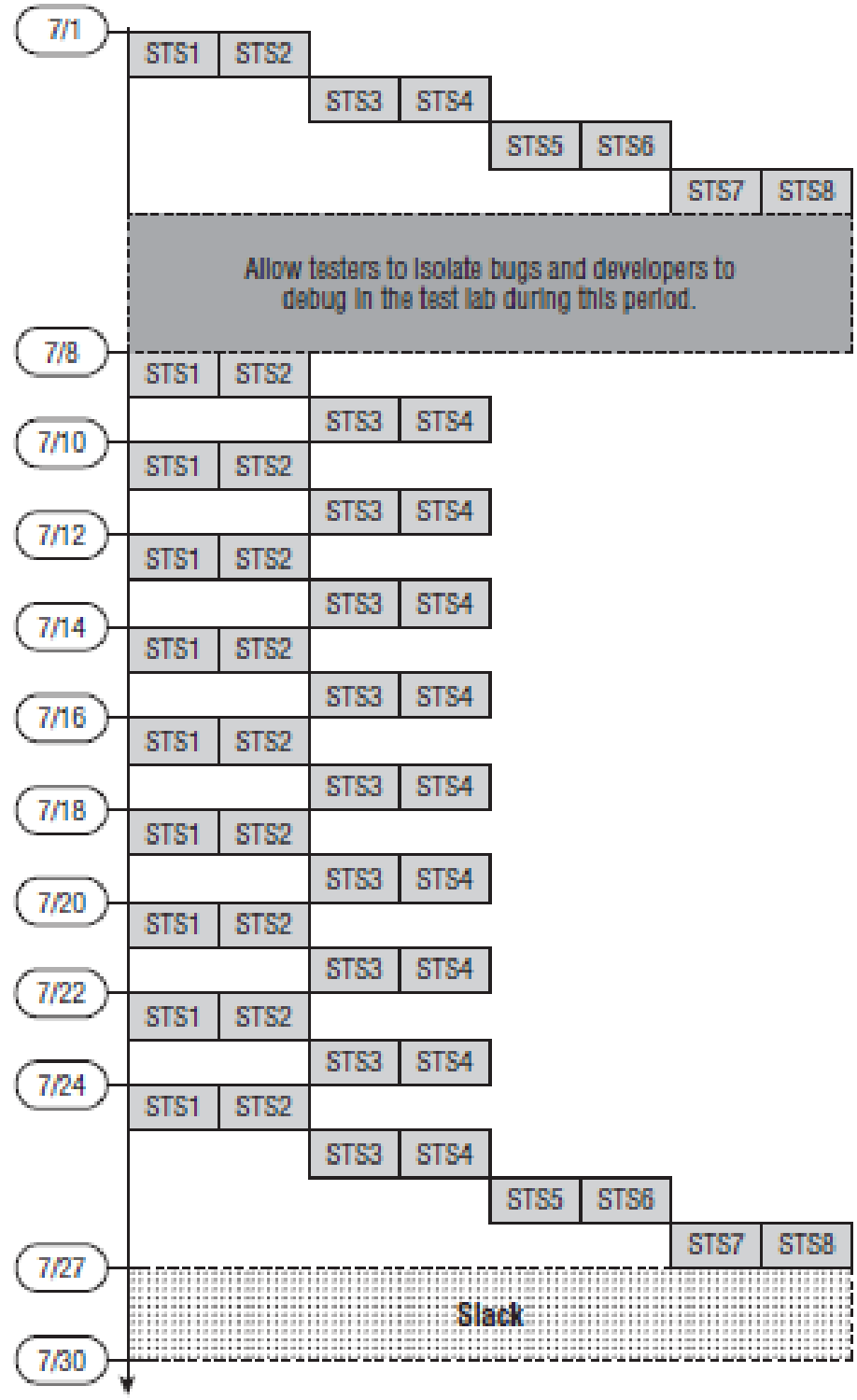
Avoiding Mistakes

- What you most want to avoid is a situation in which you never get to run an entire set of tests— or in which you run them only once early on and never return to them— because they keep getting preempted by new releases.
- This can happen if you run in test suite order, as in the railroad approach, but start over with the first test suite every time a new release shows up.
- Obviously, large regression-test gaps are created in the middle of the system test phase, while the test team repeats STS1 through STS4.
- The bill for this mistake comes due around July 27, when STS5 and STS6 are run for the first time in about 20 days.
- These tests, as well as STS7 and STS8, are likely to reveal significant problems that, because of the looming ship date, give management the unpleasant choice of delaying the product or shipping known bugs.
- People will blame the test manager for this situation, and rightly so.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

A road map for bonehead test selection



A road map for bonehead test selection

What if I Can't Repeat All the Tests? Alternative Regression Risk Mitigation Strategies

- The standard technique— you might say the brute force technique— to mitigate regression risk is to repeat the tests you have, over and over, against each release to the test team organization and against each release to the customers or users.
- Extensive automation or simple products with simple feature sets make this possible, but in some cases such test repetition is impractical.
- One of my clients tests geological modeling software, an extremely complex application with an almost unlimited potential number of data sets and workflows.
- The entire set of tests used for system test takes about two person-decades to run.
- With a 20-person test team, they have no time to repeat tests, with the exception of some very basic functionality tests that don't even come close to exercising a thousandth of the possible workflows and data sets.

What to do?

- One simple approach is, in each test cycle, to rerun every test case that failed the last time it was run, whether in the last cycle, the cycle before that, or even in the first test cycle. A slightly more risk-averse approach would be to rerun every test that has ever failed during the entire test phase during every test

“There’s a Lesson to Be Learned Here . . . ”: Test Case Incremental Improvement

- No matter how good a job you do on my first crack at developing test cases and suites, my test systems always have holes.
- Budget and time pressures, along with human mistakes, result in incomplete testing.
- Some test cases won’t cover conditions they should, or some test suites won’t include important test cases, or the overall test system won’t have all the test suites it needs.
- In the case of most consumer software and hardware products, a few omissions won’t prove fatal to the product, provided you are careful to test the critical features. (The issue of test escapes takes on a different meaning when you are dealing with safety-critical systems such as traffic-light, medical, or power-plant control computers or with situations that involve people’s privacy and security. A higher standard of quality is required in safety- or mission-critical systems.)
- Nonetheless, you can attempt to improve my test system for each subsequent release.
- A few techniques for doing this involve responding to the failures in the shipping product, incorporating the practices of others into your test system, and using exploratory testing.

Responding to Failures

- One obvious way of enhancing a test system is to plug gaps that come to light when some poor customer or user is whacked in the forehead by a test escape.
- Of course, this doesn't mean that you or anyone else in your company will celebrate test escapes. You'd rather your customers were dazzled by your product's quality.
- However, a bug found in the field is the perfect candidate for inclusion in the test system.
- You know that it's important to your customers, or to at least one of them, because they wouldn't bother to report it if it weren't.
- Assuming that development is fixing the problem, it's also important to your management.
- As an added bonus, the test case should be relatively easy to write.
- Given a field report, you should have the steps to reproduce the bug and any ancillary test data at hand.
- In addition to learning from your own test escapes, how about learning from someone else's? As the saying goes, Any fool can learn from his (or her) own errors, but a wise person learns from the mistakes of others.
- Reading trade journals, keeping up on pertinent Internet newsgroups and web sites, and networking with your peers in the test profession and your niche of the computer business will allow you to learn from their blunders.

Adopting Best Practices

- Beyond merely learning from others' mistakes, you also want to learn from their successes. I'm not suggesting industrial espionage, but rather research.

For example, simply by reading this book you are learning new test techniques, even though the book is primarily about managing testing. Likewise, trade journals might print articles that evaluate your competitors' products, often including sidebars on ''how we tested'' that you can use to benchmark your own testing.

- Training sessions, conferences, and seminars provide great opportunities to expand your test system as well as your skills.
- If you purchase test tools, you almost certainly have training options. Even if you believe that your team understands the use of a purchased tool, it might make sense for one person to attend the training session and bring back good ideas.
- Likewise, seminars are opportunities for cross-pollinating your test system with your peers.
- Every presenter at a reputable conference or training session is an expert chosen by the organizers of the event to offer unique knowledge and skills.
- Some practicing test professionals write off the academic world. It's true that research literature is not always practical. However, today's research is tomorrow's state-of-the-art product or process. I try to read at least one or two advanced books every year

Using Reactive Testing

- Speaking of reactive testing, remember that some people have a knack for rooting out errors, like a bird dog on a quail or a pig after truffles.
- Your challenge as a test manager is to convince my intrepid reactive testers to write down their brilliant ideas.
- Many of these testers tend to have a problem understanding the value of documentation.
- You, however, know when they've come up with a particularly good test: they find a bug.
- Since you capture defect data in my bug-tracking database, you can ensure that any problems found during exploratory testing turn into a documented test case.

You Can’t Do It All: Deciding What Not to Do

- As much as it pains test professionals to do so, we have to make choices about what we will and won’t test.
- With luck, you can get others to help you make those choices thereby making your life easier and providing you with some political cover at the same time.
- External support for your test goals, and the management backing it brings, is something you can’t have too much of.
- Nonetheless, at times you have to make tough decisions.
- On most projects, eventually someone asks you whether, in your professional judgment, a feature
- needs to be tested.
- What they are asking you is a question about bug coverage:

‘‘Do you think we will find critical bugs in this if we test it?’’

You Can't Do It All: Deciding What Not to Do

- There is no magical heuristic that will prevent you from making the wrong decisions.
- Hard choices are hard because no solution is obvious and the stakes are high.
- When faced with these types of choices, you can approach them with an open mind.
- You question your assumptions about what has to be tested.
- Why do you feel that way? You trust my gut, but you confirm my intuition.
- Then you talk it over.
- You ask your test team what they feel is essential, and why.
- You consult test stakeholders, peers, and colleagues.
- You quiz your development associates on the project.
- You review any pertinent references in books and articles.
- After going through this analysis process with an open mind, you make my decision.
- You document what you intend to keep and what you intend to drop.
- When you feel confident in your decision, you communicate it.
- And then you are prepared to start all over again if need be.

Case Study

In this section, you illustrate two styles of test case documentation. The first follows the screen-oriented template introduced earlier, which your team and you used for testing the browser-based program for home equity loan processing. Note that the test procedure (actions taken) is implied by the flow of the screens, but that the input data and the expected results are defined very precisely. The document, ‘‘Case Study Loan Processing Test Cases.xls,’’ is available on the basic Library page at www.rbcs-us.com. You thank your client on this project, a major midwestern bank, and the executive in charge of development, for allowing you to share this example with readers of this book. You have purged all identifying and otherwise private information from this document. Your client contact has asked that the organization remain anonymous. The second set of tests show a data-driven approach used for a custom automated test harness. This test harness sent the data to the subsystem under test— in this case, small programs running across a network providing security and access control — and then checked the return values. Note the use of HTML-style ‘‘tags’’ to define stanzas that control the harness. The documents ‘‘Case Study Functionality.suite’’ and ‘‘Case Study AddValidUser.case’’ are also available on the basic Library page at www.rbcs-us.com.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Exercises

1. You are responsible for the system test phase for a calculator program.
2. Write a test case for the multiplication feature (assume standard rather than scientific mode) in three formats:

a. The sequential step template.
b. The screen-oriented template.
c. The IEEE 829 template.
How do the three test cases compare?
The other dyadic operators (those that take two operands, like addition, division, and subtraction) can be tested using the same format. How would you change your approach for the unary operators (those that take one operand, like square root, inverse, and change sign)?
3. The expected result for a calculator must be known very precisely to ascertain pass or fail status for tests of mathematical correctness. What if we were testing the user interface (appearance, functionality, etc.) of the calculator? Can we describe precisely all the attributes that make a user interface good? Comment on whether we can describe precise pass and fail criteria for user interface tests, and whether pass or fail status is an absolute for user interface tests.